BOSTA ETL TASK

1.Introduction

This document explains the data structure, transformation logic, and database model used in the ETL job. The objective of this pipeline is to read semi-structured JSON data, flatten and transform it using PySpark, and load it into a relational data **warehouse (Amazon Redshift)** using a clean and scalable data model suitable for analytics.

2.Original JSON Data Structure

2.1 Overview

The source data is provided as a JSON file containing Airbnb listings data. Each record represents a single listing and includes both flat and nested attributes. The structure includes:

- Listing-level attributes (price, room type, accommodates, etc.)
- Nested objects (host, address, availability)
- Arrays (reviews)

This nested structure is not suitable for direct analytical querying in a relational database, which is why flattening and normalization are required.

2.2 Sample Original JSON (Excerpt)


```
{
    "_id": "1255979",
    "listing_url": "https://www.airbnb.com/rooms/1255979",
    "name": "CHARMING PRACTICAL RELAXING IPANEMA",
    "summary": "CHARMING QUIET SAFE PRACTICAL 1 double bed + 1 single mattress if needed  WIFI - TV -
    "space": "Very Nice and Charmous Flat, with one bed room, american kitchen, quiet and very safe!.
    "description": "CHARMING QUIET SAFE PRACTICAL 1 double bed + 1 single mattress if needed  WIFI -
    "neighborhood_overview": "Nicest place in Rio to stay. Safe, nature closed everywhere, transporta
    "notes": "It a very real and charming place! Very good energy!",
    "transit": "All transports closed, Bus, Metro, Taxi, Bikes...",
    "access": "Only the laundry space will be unavailable.",
    "interaction": "You can count on me for an effective  comunication by E-Mail, (SENSITIVE CONTENTS
    "house_rules": "Prendre soin de l apartement bien sûr ( c est chez moi, pas un hotel!). Eviter d
    "property_type": "Apartment",
    "room_type": "Entire home/apt",
    "bed_type": "Real Bed",
    "minimum_nights": "2",
    "maximum_nights": "120",
    "cancellation_policy": "moderate",
    "last_scraped": {
        "$date": {
            "$numberLong": "1549861200000"
        }
    },
    "calendar_last_scraped": {
        "$date": {
            "$numberLong": "1549861200000"
        }
    },
    "first_review": {
        "$date": {
            "$numberLong": "1388725200000"
        }
```


3. Flattened Data Output

3.1 Flattening Logic

The JSON file is read using **PySpark** with multiline support enabled. Nested structures are flattened using:

- Column selection for nested structs (e.g. host.host_id)
- explode() function for arrays such as reviews

Each flattened dataset is written independently to represent a table in the target database.

4. ER Diagram and Data Model

4.1 Objective

The objective of the database model design is to create a normalized, scalable, and analytics-ready schema that ensures data integrity, avoids redundancy, and provides clear, maintainable relationships between entities.

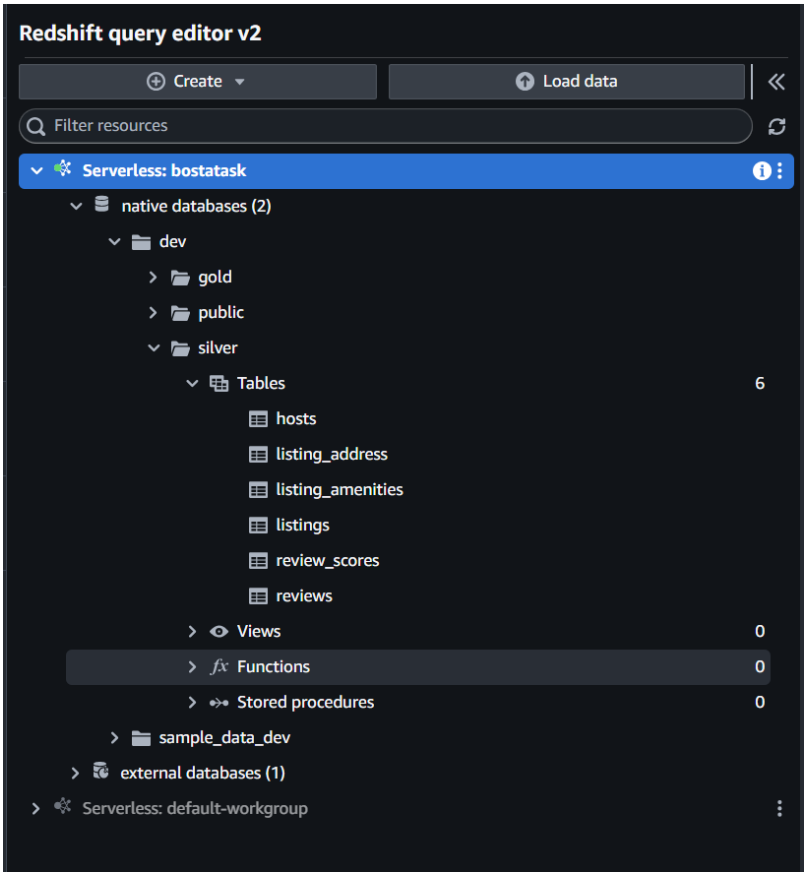
4.2 Architectural Approach: **Medallion Architecture**

The data warehouse implementation follows the Medallion Architecture pattern, which organizes data into progressive layers based on data quality and business readiness.

In this project, two layers were implemented:

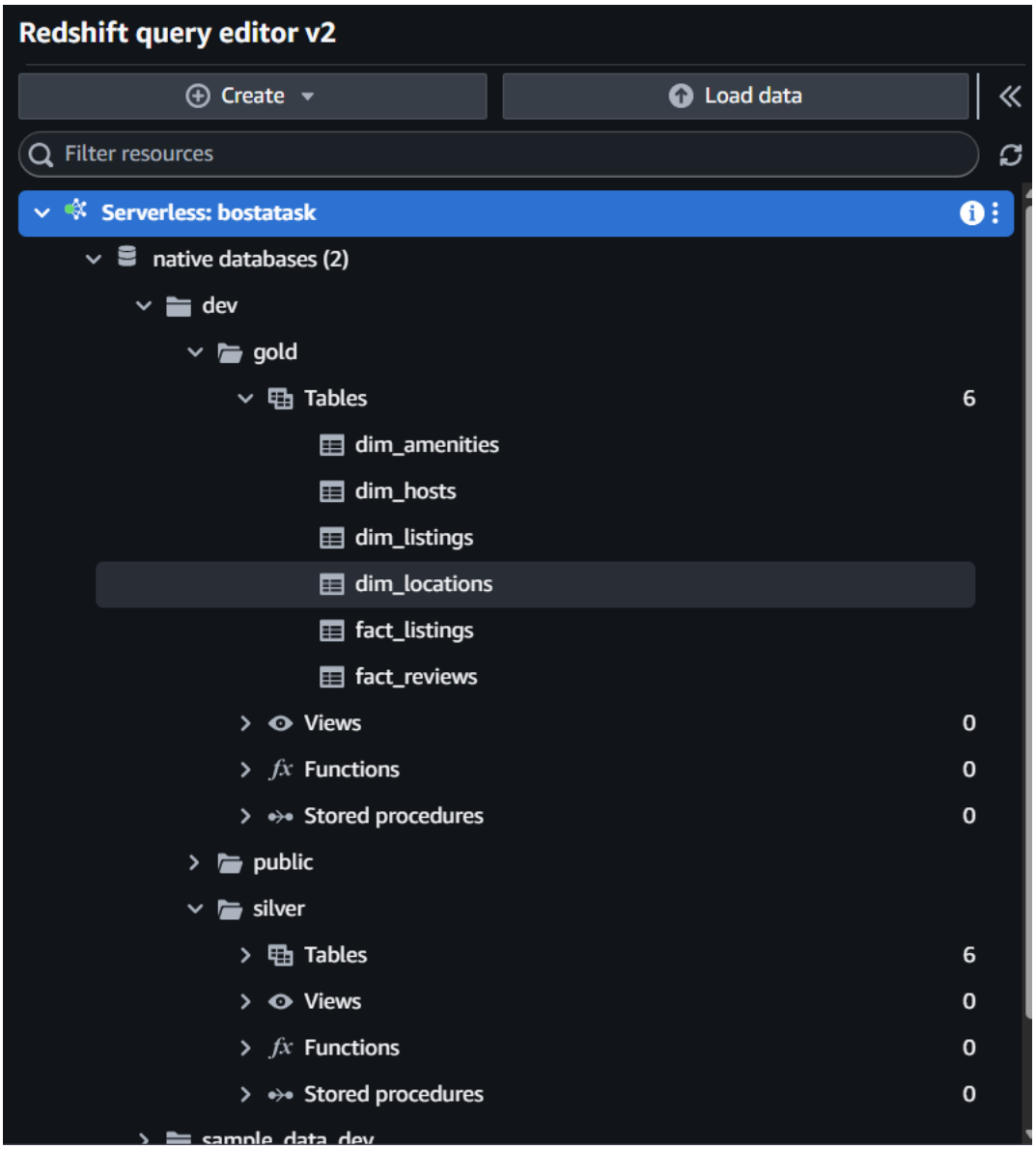
4.2.1 Silver Layer:

- Represents cleaned and standardized data after flattening the raw JSON
- Contains structured tables such as listings, hosts, addresses, and reviews
- Data types are enforced and nested structures are removed
- Used as the foundation for downstream analytical modeling



4.2.2 Gold Layer

- Represents business-ready, analytics-focused data
- Contains fact and dimension tables modeled using a Star Schema
- Optimized for reporting, aggregation, and BI consumption



4.3 Why Medallion Architecture?

The Medallion Architecture was chosen because it:

- Enforces a clear separation of concerns
- Improves data quality progressively

- Simplifies debugging and reprocessing
- Aligns with modern cloud data engineering best practices

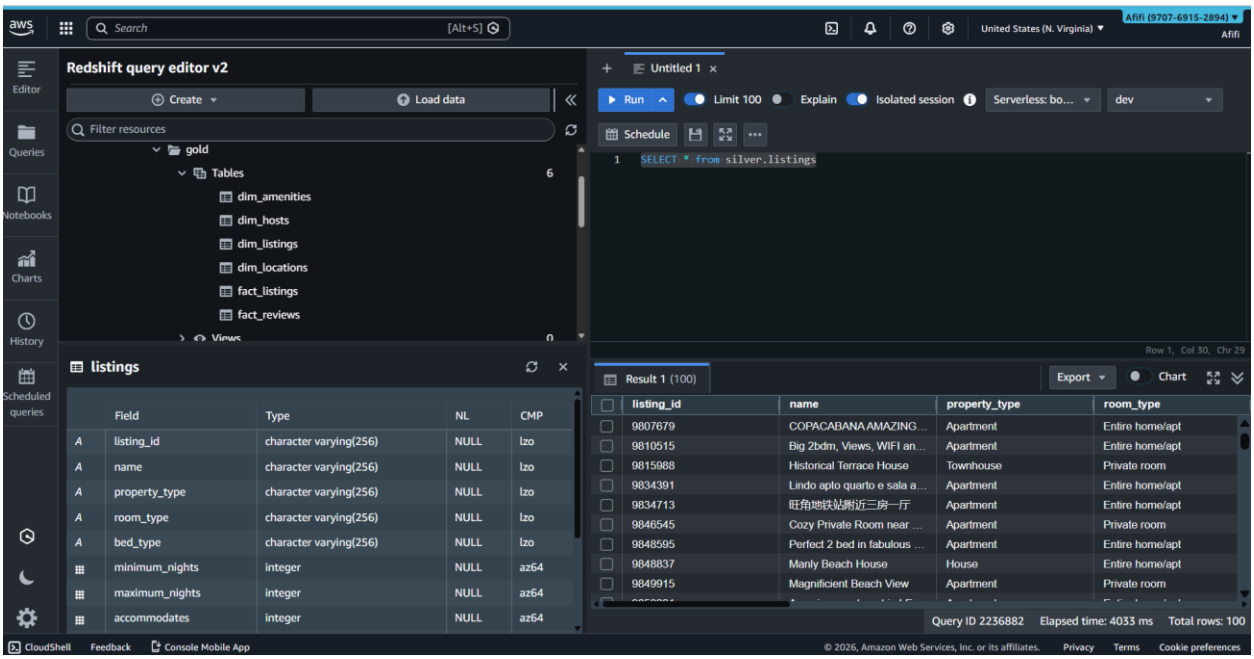
4.3.1 Why Star Schema in the Gold Layer?

A Star Schema was chosen for the following reasons:

- Optimized for analytical queries involving aggregations
- Minimizes join complexity, resulting in faster query execution
- Improves query performance on Redshift
- High compatibility with Amazon Redshift's columnar architecture
- Improved readability and usability for BI and reporting tools
- Scalable and extensible design
- Improved query performance
- Reduced storage redundancy
- Simplified maintenance
- Analytics-ready schema

4.4 Data Validation Example

To validate the model and data loading process, a sample query was executed against the listings table in the Silver layer.



A screenshot showing the contents of the listings table in Amazon Redshift is included to demonstrate:

- Correct data ingestion
- Proper schema enforcement
- Readiness for analytical querying

4.5 Design Approach

The database follows a Star Schema design to support analytical workloads. This approach:

- Separates facts from dimensions
- Avoids nested or semi-structured data in the warehouse
- Improves query performance and readability

4.5.1 Benefits of the Chosen Model

- Clear data lifecycle from ingestion to analytics

- Reduced data redundancy
- Improved query performance
- Maintainable and scalable warehouse design

4.6 Tables Overview

Dimensions

- DIM_HOST
- DIM_ADDRESS
- DIM_PROPERTY
- DIM_REVIEWER
- DIM_DATE

Facts

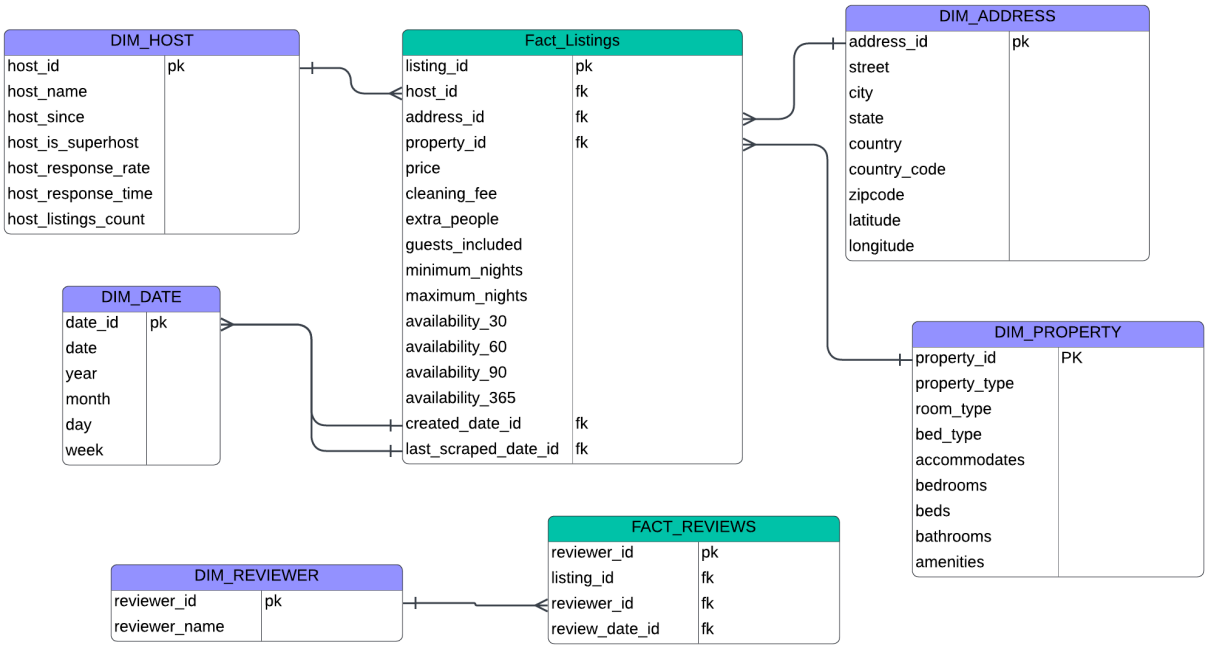
- FACT_LISTINGS
- FACT_REVIEWS

4.6 Relationships

- One Host can manage multiple Listings
- Each Listing is associated with one Address and one Property configuration
- Each Listing can have multiple Reviews
- Reviews are linked to Reviewers and Dates

This design ensures strong referential integrity and eliminates data duplication.

4.7 ER Diagram



5. Enhancing Query Performance

5.1 Objective

The objective of this section is to optimize query performance for analytical workloads executed on Amazon Redshift.

5.2 Optimization Techniques

- Star Schema Design
  - Minimizes join complexity
  - Enables efficient aggregation queries
- Columnar Storage
  - Redshift scans only required columns
  - Reduces I/O and execution time
- Distribution Keys
  - Shared distribution keys between fact and dimension tables
  - Minimizes data movement across nodes
- Sort Keys

    - Applied on frequently filtered columns such as listing_id and review_date
    - Improves range and time-based queries
- Data Type Optimization
    - Appropriate data types used to minimize storage and processing overhead

6. Key Variables Explanation

6.1 Business-Critical Fields

- **listing_id:** Unique identifier for each property listing
- **host_id:** Identifies the host associated with a listing
- **price:** Nightly price of the listing (normalized to numeric format)
- **availability_30:** Availability in the next 30 days
- **review_id:** Unique identifier for each review

6.2 Why These Variables Matter

These variables are essential for answering analytical questions such as:

- Host performance and listing distribution
- Pricing analysis by location and property type
- Availability trends
- Customer sentiment analysis using reviews

7 Notification System for Job Failures

7.1 Objective

To ensure operational reliability, an automated email-based notification system was implemented to alert stakeholders when the ETL job fails.

7.2 Design Overview

The ETL job is executed locally using PySpark and wrapped in exception handling logic. Upon failure, an email notification is sent to a Gmail address.

7.3 Implementation Logic

- ETL execution is enclosed within a try-except block
- On exception, the error message is captured
- An email is triggered using Python's smtplib

```python

SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 465

SENDER_EMAIL = "ahmed.khaled.afifi22@gmail.com"
APP_PASSWORD = "xanjopzllovfjiyf"
RECEIVER_EMAIL = "ahmed.khaled.afifi22@gmail.com"


def send_failure_email(error):
    msg = EmailMessage()
    msg["Subject"] = "Spark Job Failed"
    msg["From"] = SENDER_EMAIL
    msg["To"] = RECEIVER_EMAIL

    msg.set_content(
        f"""
Bosta Spark Job FAILED.

Error:
{str(error)}
"""
    )

    try:
        with smtplib.SMTP_SSL(SMTP_SERVER, SMTP_PORT) as server:
            server.login(SENDER_EMAIL, APP_PASSWORD)
            server.send_message(msg)
    except Exception:
        logging.exception("Failed to send failure email")

```

7.4 Notification Details

The failure notification includes:

- Job name
- Timestamp
- Error details

Spark Job Failed Inbox ×

ahmed.khaled.afifi22@gmail.com 4:37 AM (12 hours ago)
to me

Bosta Spark Job FAILED.

Error:
Column host_total_listings_count not found in schema Some(StructType(StructField(host_id,StringType,true),StructField(host_name,StringType,true),StructField(host_is_superhost,BooleanType,true),StructField(host_response_time,StringType,true),StructField(host_response_rate,IntegerType,true),StructField(host_listings_count,IntegerType,true),StructField(host_location,StringType,true))).

7.5 Benefits

- Immediate failure awareness
- Faster troubleshooting
- No dependency on external orchestration tools

Future Enhancements

In production environments, this solution can be extended to **Airflow,** centralized logging, and monitoring dashboards.

Code Structure and Modularity Design

Objective

The objective of this section is to explain the structure of the ETL codebase and how modular design principles were applied to ensure maintainability, clarity, and ease of extension.

The ETL solution was intentionally designed as a **modular Python project** rather than a single monolithic script. Each file has a clear responsibility, which aligns with production-grade data engineering best practices.

Codebase Organization Overview

The codebase is organized into multiple Python modules, each handling a specific concern such as configuration management, Spark initialization, data transformation, orchestration, and failure notification.

This separation of concerns:

- Improves readability
- Simplifies debugging
- Allows independent modification of components
- Makes the pipeline easier to extend in the future

File Responsibilities

config.py

This file contains all configuration-related parameters, including:

- File paths
- Schema names
- Database connection details
- Environment-specific settings

Centralizing configurations avoids hardcoding values across the codebase and allows easier changes between environments.

spark_session.py

This module is responsible for creating and configuring the SparkSession.
It ensures that:

- Spark is initialized consistently
- Configuration changes are applied in one place
- Spark-related setup is decoupled from business logic

transformation.py

This file contains all data transformation logic, including:

- Flattening nested JSON structures
- Casting data types
- Selecting and renaming columns
- Preparing data for analytical consumption

Separating transformations into a dedicated module keeps the business logic clean and reusable.

spark_job.py

This module orchestrates the ETL execution flow.
It is responsible for:

- Calling the Spark session
- Executing transformation functions
- Writing processed data to Amazon Redshift

It acts as the core ETL pipeline controller without containing low-level logic.

email_notification.py

This module handles operational notifications.
It is used to:

- Send automated email alerts on job failure
- Include error details for troubleshooting

This improves observability and ensures failures are detected immediately.

main.py

This is the entry point of the application.
It:

- Triggers the ETL job
- Wraps execution in exception handling
- Invokes the email notification mechanism upon failure

This design ensures that orchestration and error handling are clearly separated from transformation logic.